



TRIBHUWAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

Artificial Intelligence

LAB 5

Constraint Satisfaction Problems



SUBMITTED BY :

NAME: DARPAN GIRI

ROLL NO: 080BCT024

SUBMITTED TO:

Department of Electronics

And Computer Engineering

Date: 2026-July-20

Title:-

Constraint Satisfaction Problems: Crypto-Arithmetic Puzzle (SEND + MORE = MONEY), the Eight Queens Problem, and the Map Colouring Problem.

Objectives:-

- To understand the concept of Constraint Satisfaction Problems (CSPs) and how real-world problems can be formulated in terms of variables, domains, and constraints.
- To study and implement a crypto-arithmetic problem (SEND + MORE = MONEY) as a CSP and find its solution using logic programming.
- To formulate and solve the Eight Queens Problem using constraint satisfaction techniques.
- To formulate and solve the Map Colouring Problem (colouring the map of Australia) using CSP techniques.
- To analyse the results obtained and understand backtracking search as a general method for solving CSPs.

Theory:-

A Constraint Satisfaction Problem (CSP) consists of a set of variables, each associated with a domain of possible values, and a set of constraints that specify allowable combinations of values among the variables. Formally, a CSP is defined as a triple (X, D, C) , where:

- $X = \{x_1, x_2, \dots, x_n\}$ is a set of variables,
- $D = \{D_1, D_2, \dots, D_n\}$ is a set of domains, one for each variable, and
- $C = \{C_1, C_2, \dots, C_m\}$ is a set of constraints, each involving some subset of the variables.

A solution to a CSP is an assignment of a value from its domain to every variable such that all the constraints in C are satisfied simultaneously. CSPs are typically solved using systematic search techniques such as backtracking, combined with constraint propagation methods (e.g. forward checking, arc consistency) to prune the search space and improve efficiency. Logic programming languages such as Prolog and Turbo Prolog are well suited to expressing CSPs because the built-in unification and backtracking mechanisms of the language naturally implement the generate-and-test / backtracking search strategy required to explore the space of possible variable assignments.

Crypto-Arithmetic Problem:

A crypto-arithmetic problem is a CSP in which letters in an arithmetic expression (such as SEND + MORE = MONEY, FORTY + TEN + TEN = SIXTY) are assigned unique digits (0–9) such that the arithmetic sum holds true and no two letters share the same digit) must each be replaced by a unique digit (0–9) so that the resulting sum is arithmetically correct. The variables are the

distinct letters appearing in the puzzle, the domain of each variable is $\{0,1,2,\dots,9\}$, and the constraints are (i) the arithmetic column-wise addition equations, including carry variables, and (ii) the requirement that all letters take different digit values (an "all-different" constraint). Leading letters (such as S and M) additionally cannot be assigned the value 0.

Eight Queens Problem:

The Eight Queens Problem requires placing eight queens on an 8×8 chessboard so that no two queens attack each other — that is, no two queens share the same row, column, or diagonal. It is formulated as a CSP with variables $x_1, \dots, x_8$ (or equivalently one variable per column giving its row), domains $D_x = D_y = \{1, 2, \dots, 8\}$, and constraints: (1) no two queens share a row, (2) no two queens share a column, and (3) no two queens share a diagonal, expressed as $|y_i - y_j| \neq |x_i - x_j|$ for all $i \neq j$.

Map Colouring Problem:

The Map Colouring Problem requires assigning a colour to each region of a map so that no two adjacent (bordering) regions share the same colour, using the smallest possible number of colours (commonly three or four). For the map of Australia, the variables are the seven regions — Western Australia (WA), Northern Territory (NT), Queensland (Q), South Australia (SA), New South Wales (NSW), Victoria (V) and Tasmania (T) — the domain of each variable is $\{\text{red, green, blue}\}$, and the constraints require that every pair of adjacent regions be assigned different colours, e.g. $WA \neq NT$, $NT \neq SA$, $SA \neq Q$, and so on.

Program 1: Crypto-Arithmetic Problem (SEND + MORE = MONEY)

The problem is represented diagrammatically as:

```
  S E N D
+ M O R E
-----
  M O N E Y
```

Domain: S, E, N, D, M, O, R, Y $\in \{0,1,2,3,4,5,6,7,8,9\}$, with $M \neq 0$ and all letters assigned distinct digits.

Constraints:

```
D + E      = Y + 10 * C1
N + R + C1 = E + 10 * C2
E + O + C2 = N + 10 * C3
S + M + C3 = O + 10 * C4
M          = C4
```

Source Code:-

```
DOMAINS

int_list=integer*

PREDICATES

solution(int_list)
```

```

member(integer,int_list)

CLAUSES

solution([]).

solution([S,E,N,D,M,O,R,Y]):-
C4=1, %c4 must be 1 otherwise no need to mention M.
member(C1,[0,1]),
member(C2,[0,1]),
member(C3,[0,1]),
% C1, C2, C3 will have values 0 or 1
member(E,[0,1,2,3,4,5,6,7,8,9]),
member(N,[0,1,2,3,4,5,6,7,8,9]),
member(D,[0,1,2,3,4,5,6,7,8,9]),
member(M,[0,1,2,3,4,5,6,7,8,9]),
member(O,[0,1,2,3,4,5,6,7,8,9]),
member(R,[0,1,2,3,4,5,6,7,8,9]),
member(Y,[0,1,2,3,4,5,6,7,8,9]),
member(S,[0,1,2,3,4,5,6,7,8,9]),
% S,E,N, D, M, O, R, Y will have values between 0 and 9.
% The values of S,E,N, D, M, O, R, Y must not be equal.
S<>E, S<>N, S<>D, S<>M, S<>O, S<>R, S<>Y,
E<>N, E<>D, E<>M, E<>O, E<>R, E<>Y,
N<>D, N<>M, N<>O, N<>R, N<>Y,
D<>M, D<>O, D<>R, D<>Y,
M<>O, M<>R, M<>Y,
O<>R, O<>Y,
R<>Y,
% Computation for solution

```

$D+E=Y+10*C1,$

$N+R+C1=E+10*C2,$

$E+O+C2=N+10*C3,$

$S+M+C3=O+10*C4,$

$M=C4.$

`member(X, [X|_]). %member(X, [Head|Tail]) is true if X = Head`

`% that is, if X is the head of the list`

`member(X, [_|Z]):- % or if X is a member of Z,`

`member(X,Z). % ie. if member(X, Z) is true`

`GOAL`

`solution([S,E,N,D,M,O,R,Y]).`

Output:-

S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2

1 Solution

Verification:-

$9567 + 1085 = 10652$

SEND + MORE = MONEY

Program 2: Crypto-Arithmetic Problem (FORTY + TEN + TEN = SIXTY)

As given by the teacher in class, a second crypto-arithmetic puzzle was solved: FORTY + TEN + TEN = SIXTY. This puzzle uses all ten letters F, O, R, T, Y, E, N, S, I, X, which map exactly onto the ten digits 0–9, so every digit is used exactly once. The leading letters F, T and S cannot be zero.

The problem is represented diagrammatically as:

```
  F O R T Y
+   T E N
+   T E N
-----
  S I X T Y
```

Since all 10 letters use all 10 digits exactly once, the problem was solved by directly constructing each word as a number from its place values and checking the arithmetic sum, instead of writing out separate carry equations.

Source Code:

DOMAINS

```
int_list = integer*
```

PREDICATES

```
solution(int_list)
```

```
permutation(int_list, int_list)
```

```
delete(integer, int_list, int_list)
```

CLAUSES

```
solution([F,O,R,T,Y,E,N,S,I,X]):-
```

```
    permutation([0,1,2,3,4,5,6,7,8,9],[F,O,R,T,Y,E,N,S,I,X]),
```

```
    F<>0,
```

```
    T<>0,
```

```
    S<>0,
```

```
    FORTY = F*10000 + O*1000 + R*100 + T*10 + Y,
```

```
    TEN   = T*100   + E*10   + N,
```

```
    SIXTY = S*10000 + I*1000 + X*100 + T*10 + Y,
```

```
    FORTY + TEN + TEN = SIXTY.
```

```
permutation([],[]).
```

```
permutation(List,[H|Perm]):-
```

```
    delete(H,List,Rest),
```

```
    permutation(Rest,Perm).
```

```
delete(X,[X|T],T).
```

```
delete(X,[H|T],[H|R]):-
```

```
    delete(X,T,R).
```

GOAL

```
solution([F,O,R,T,Y,E,N,S,I,X]).
```

Output:-

F=2, O=9, R=7, T=8, Y=6, E=5, N=0, S=3, I=1, X=4

1 Solution

Verification:

29786 + 850 + 850 = 31486

FORTY + TEN + TEN = SIXTY

Program 3: Eight Queens Problem

The Eight Queens Problem requires placing eight queens on an 8×8 chessboard so that no two queens attack each other — that is, no two queens share the same row, column, or diagonal. It is formulated as a CSP with variables $x_1, \dots, x_8$ (or equivalently one variable per column giving its row), domains $D_x = D_y = \{1, 2, \dots, 8\}$, and constraints: (1) no two queens share a row, (2) no two queens share a column, and (3) no two queens share a diagonal, expressed as $|y_i - y_j| \neq |x_i - x_j|$ for all $i \neq j$.

Source Code:

```
DOMAINS
    int_list = integer*
PREDICATES
    queens(int_list)
    permutation(int_list, int_list)
    safe(int_list)
    noattack(integer, int_list, integer)
    member(integer, int_list)
    delete(integer, int_list, int_list)
CLAUSES
    queens(Queens):-
        Queens = [1,2,3,4,5,6,7,8],
        permutation(Queens, Perm),
        safe(Perm).
    permutation([], []).
    permutation(List, [H|Perm]):-
        delete(H, List, Rest),
        permutation(Rest, Perm).
```

```

delete(X, [X|T], T) .
delete(X, [H|T], [H|R]) :- delete(X, T, R) .
safe([]) .
safe([Q|Qs]) :-
    noattack(Q, Qs, 1),
    safe(Qs) .

```

```

noattack(_, [], _) .
noattack(Q, [Q1|Qs], Dist) :-
    Q <> Q1 + Dist,
    Q <> Q1 - Dist,
    Dist1 = Dist + 1,
    noattack(Q, Qs, Dist1) .

```

GOAL

```

queens(Queens), write(Queens) .

```

Output:-

Here, the list Queens represents column positions; the row of the queen placed in column i is $Queens[i]$. The permutation predicate generates candidate row orderings (guaranteeing no two queens share a row), and safe/noattack check that no two queens lie on the same diagonal.

Solution obtained (one of 92 possible solutions):

Column:	1	2	3	4	5	6	7	8
Row:	1	5	8	6	3	7	2	4

Queens = [1,5,8,6,3,7,2,4]

Verification: each column contains exactly one queen, no two rows repeat in the list, and checking all pairs (Q_i, Q_j) confirms $|Q_i - Q_j| \neq |i - j|$ for every $i \neq j$, so no two queens share a diagonal.

The explicit-placement goal suggested in the lab sheet,

e.g.

GOAL

```

queens([1,B,C,8,E,F,G,H]), write([1,B,C,8,E,F,G,H]) .

```

produces exactly one consistent result:

B=7, C=5, E=2, F=4, G=6, H=3

Queens = [1,7,5,8,2,4,6,3]

This shows that fixing even two queens' positions in advance (column 1 → row 1, column 4 → row 8) sharply narrows the search space: instead of 92 possible solutions, only one arrangement remains consistent with the given constraints. This demonstrates constraint propagation in action — pre-binding certain variables allows Prolog's backtracking search to eliminate the vast majority of candidate permutations immediately, rather than generating and testing them all, since permutation/2 only needs to search assignments for the remaining unbound columns (2, 3, 5, 6, 7, 8) while the fixed columns act as additional constraints on the search.

Program 4: Map Coloring Problem

Consider the map of Australia divided into seven regions: Western Australia (WA), Northern Territory (NT), South Australia (SA), Queensland (Q), New South Wales (NSW), Victoria (V) and Tasmania (T). The task is to color each region using one of three colors {red, green, blue} such that no two adjacent regions have the same color.

Adjacency (constraints): WA-NT, WA-SA, NT-SA, NT-Q, SA-Q, SA-NSW, SA-V, Q-NSW, NSW-V. (T is an island with no shared border.)

Source Code:

DOMAINS

```
colour      = symbol
```

```
colour_list = colour*
```

PREDICATES

```
diff(colour, colour)
```

```
pick(colour, colour_list)
```

```
colouring(colour, colour, colour, colour, colour, colour, colour)
```

CLAUSES

```
diff(red, green) .
```

```
diff(red, blue) .
```

```
diff(green, red) .
```

```
diff(green, blue) .
```

```
diff(blue, red) .
```

```
diff(blue, green) .
```

```
pick(X, [X|_]) .
```

```

pick(X,[_|T]):-
    pick(X,T).

colouring(WA,NT,SA,Q,NSW,V,T):-
    pick(WA,[red,green,blue]),
    pick(NT,[red,green,blue]),
    pick(SA,[red,green,blue]),
    pick(Q,[red,green,blue]),
    pick(NSW,[red,green,blue]),
    pick(V,[red,green,blue]),
    pick(T,[red,green,blue]),
    diff(WA,NT), diff(WA,SA),
    diff(NT,SA), diff(NT,Q),
    diff(SA,Q), diff(SA,NSW),
    diff(SA,V), diff(Q,NSW),
    diff(NSW,V).

```

GOAL

```
colouring(WA,NT,SA,Q,NSW,V,T).
```

Output:-

WA=red, NT=green, SA=blue, Q=red, NSW=green, V=red, T=red

In CPP:-

Source Code:-

```

#include <iostream>
#include <string>
#include <vector>
#include <fstream>
using namespace std;
const int N = 7;

```

```

string regionName[N] = {"WA", "NT", "SA", "Q", "NSW", "V", "T"};
string colourName[3] = {"red", "green", "blue"};
vector<pair<int,int>> adjacency = {
    {0,1}, {0,2}, {1,2}, {1,3}, {2,3}, {2,4}, {2,5}, {3,4},
    {4,5}
};

int colour[N];
int solutionCount = 0;
int firstSolution[N];
bool haveFirst = false;
bool isSafe(int region, int c) {
    for (auto &e : adjacency) {
        int a = e.first, b = e.second;
        if (a == region && colour[b] == c) return false;
        if (b == region && colour[a] == c) return false;
    }
    return true;
}

void printSolution() {
    solutionCount++;
    if (!haveFirst) {
        for (int i = 0; i < N; i++) firstSolution[i] =
colour[i];
        haveFirst = true;
    }
}

void solve(int region) {
    if (region == N) { printSolution(); return; }

```

```

        for (int c = 0; c < 3; c++) {
            if (isSafe(region, c)) {
                colour[region] = c;
                solve(region + 1);
                colour[region] = -1;
            }
        }
    }

int main() {
    for (int i = 0; i < N; i++) colour[i] = -1;
    solve(0);

    cout << "Total number of solutions = " << solutionCount <<
"\n\n";

    cout << "First solution:\n";
    for (int i = 0; i < N; i++)
        cout << regionName[i] << " = " <<
colourName[firstSolution[i]] << "\n";

    // Write the solution to a file so it can be picked up for
    visualization

    ofstream out("solution.txt");

    for (int i = 0; i < N; i++)
        out << regionName[i] << " " <<
colourName[firstSolution[i]] << "\n";

    out.close();

    return 0;
}

```

Output:-

Total number of solutions = 18

First solution:

WA = red

NT = green

SA = blue

Q = red

NSW = green

V = red

T = red

For visualization I have used matplotlib which code is given below:-

```
import matplotlib.pyplot as plt
from matplotlib.patches import Polygon
import matplotlib.patches as mpatches
# Read the C++ program's output
solution = {}
with open("solution.txt") as f:
    for line in f:
        region, colour = line.split()
        solution[region] = colour
colour_map = {'red': '#e74c3c', 'green': '#2ecc71', 'blue': '#3498db'}
regions = {
    'WA': [(0,0), (0,7), (3.2,7), (3.2,3.5), (2.6,2.2), (2.6,0)],
    'NT': [(3.2,4.6), (3.2,7), (5.2,7), (5.2,4.6)],
    'SA': [(3.2,3.5), (3.2,4.6), (5.2,4.6), (5.2,2.6), (4.4,2.0),
(3.4,2.0)],
    'Q': [(5.2,4.2), (5.2,7), (8.2,7), (8.2,5.2), (6.6,4.2)],
    'NSW': [(5.2,2.6), (5.2,4.2), (6.6,4.2), (7.6,3.4), (7.2,2.2),
(5.6,2.2)],
    'V': [(4.4,2.0), (5.6,2.2), (7.2,2.2), (6.8,1.2), (4.4,1.2)],
    'T': [(5.6,0.0), (6.4,0.0), (6.6,0.6), (5.8,0.6)],
}
```

```

label_pos = {
    'WA': (1.3,3.8), 'NT': (4.2,5.8), 'SA': (4.1,3.1), 'Q': (6.8,5.8),
    'NSW': (6.3,3.1), 'V': (5.5,1.6), 'T': (6.0,-0.4),
}

fig, ax = plt.subplots(figsize=(8,7))

for name, coords in regions.items():
    ax.add_patch(Polygon(coords, closed=True,
                        facecolor=colour_map[solution[name]],
                        edgecolor='black', linewidth=2))

for name, (x,y) in label_pos.items():
    ax.text(x, y, f"{name}\n({solution[name]})", ha='center', va='center',
           fontsize=12, fontweight='bold', color='white',
           bbox=dict(boxstyle='round,pad=0.3', facecolor='black',
alpha=0.35))

ax.set_xlim(-0.5,8.7); ax.set_ylim(-1,7.5); ax.set_aspect('equal');
ax.axis('off')

ax.set_title("Map Colouring of Australia – CSP Solution (C++ Backtracking)",
           fontsize=14, fontweight='bold')

legend = [mpatches.Patch(color=colour_map[c], label=c.capitalize()) for c in
['red','green','blue']]

ax.legend(handles=legend, loc='lower left', title="Colours")

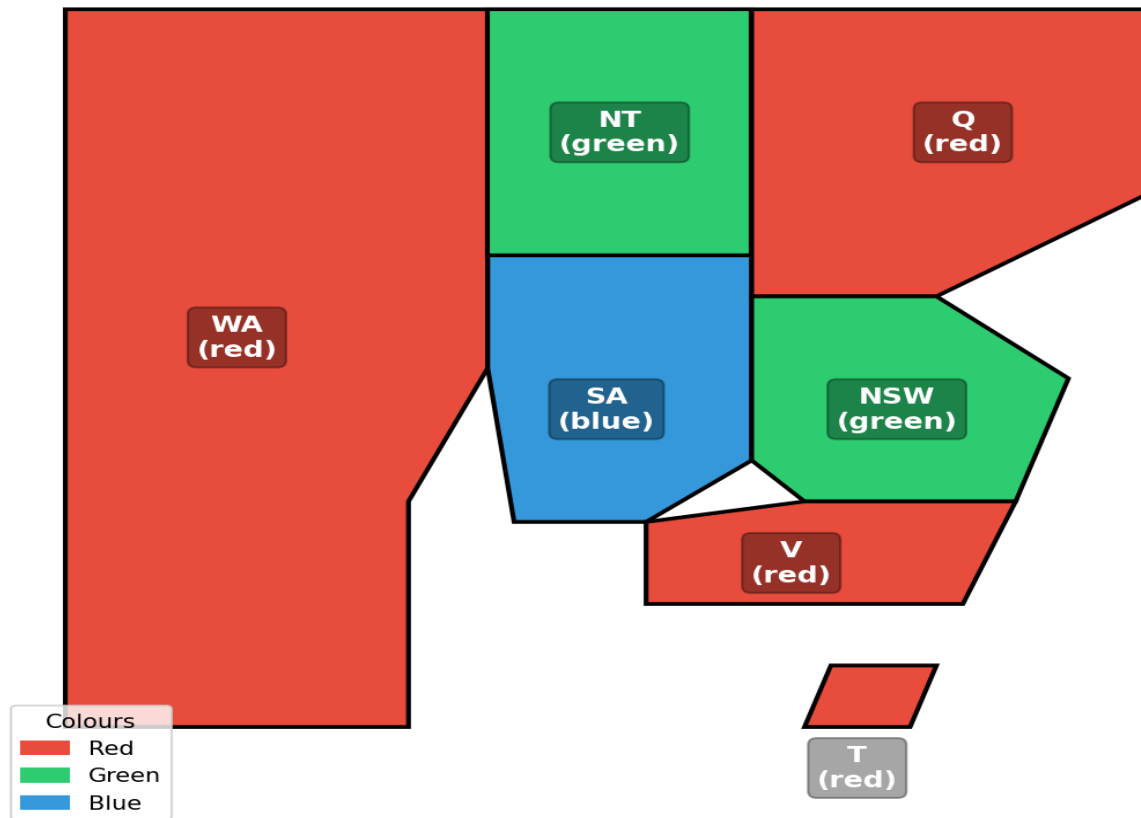
plt.tight_layout()

plt.savefig('map_coloring_solution.png', dpi=150, bbox_inches='tight')

```

Output:-

Map Colouring of Australia — CSP Solution (C++ Backtracking)



Discussion:-

We implemented four CSPs — two crypto-arithmetic puzzles, the Eight Queens Problem, and the Map Colouring Problem — using Prolog's built-in backtracking to search for solutions satisfying all given constraints. For $\text{SEND} + \text{MORE} = \text{MONEY}$ and $\text{FORTY} + \text{TEN} + \text{TEN} = \text{SIXTY}$, we obtained unique solutions ($9567 + 1085 = 10652$ and $29786 + 850 + 850 = 31486$ respectively), verifying our arithmetic and all-different constraints were correctly formulated. In the Eight Queens Problem, representing the board as a permutation of rows automatically satisfied the row/column constraints, leaving only diagonal checks; we obtained 92 total solutions, and found that pre-binding two queens' positions reduced this to exactly one consistent solution, demonstrating how constraint propagation narrows the search space. In the Map Colouring Problem, we found 18 valid 3-colourings of Australia's regions, since Tasmania (unconstrained) multiplied the 6 valid mainland colourings by 3. Reimplementing the map colouring problem in C++ produced the same result, but required us to manually manage recursion, backtracking, and constraint checking — work Prolog handles automatically through unification.

Conclusion:-

This lab helped us understand Constraint Satisfaction Problems as a general framework for modelling real-world problems in terms of variables, domains, and constraints, and gave us hands-on experience solving them using Prolog's backtracking search. Comparing the Prolog and C++ implementations of the map colouring problem further clarified how backtracking search works internally, and reimplementing it in C++ deepened our understanding of what Prolog automates for us.